XLM-R Vector Generation and Visualization Toolkit

Data Engineering Team

June 18, 2026

Overview

This document contains a comprehensive toolkit of scripts utilized for generating vector embeddings from the Kurdish medical dataset using the XLM-R model, and visualizing the semantic relationships between these embeddings. Below is a summary table containing links to the code for each step in the pipeline, alongside an explanation of its purpose.

Script Name	Section Link	Purpose / Why We Use It
Vector Pipeline		
Vector Generator	Section 1	Generates vector embeddings for a batch of texts using the XLM-R large model.
Vector Visualization	Section 2	Computes cosine similarity between vectors and builds an interactive pyvis network graph.
Setup & Environment		
Setup Script	Section 3	Prepares the environment by installing dependencies and caching the XLM-R model.
Model Loader	Section 4	A dedicated script to trigger the Hugging Face download of the XLM-R model for caching.
Requirements	Section 5	Lists the necessary Python packages for the pipeline.
Environment Variables	Section 6	Configuration file for specifying model and paths.

Table 1: Vector generation and setup scripts categorized by their functional domain.

1 Vector Generator

Purpose: We use this script to generate vector embeddings for the Kurdish medical dataset using the `xlm-roberta-large` model. It includes batching, checkpointing, and GPU acceleration.

Listing 1: vector_generator.py

```python
import json
import os
import logging
from typing import List, Dict
from tqdm import tqdm
import torch
from transformers import AutoTokenizer, AutoModel

# Setup Logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler("vector_pipeline.log"),
        logging.StreamHandler()
    ]
)

# Configuration
# We use xlm-roberta-base (or xlm-roberta-large) as requested
MODEL_NAME = "xlm-roberta-large"
DATASET_PATH = "../just_data_vector/kurdish_medical_corpus_kmc.json"
OUTPUT_PATH = "kurdish_medical_vectors.jsonl"
CHECKPOINT_PATH = "processed_ids.txt"

class VectorGenerator:
    def __init__(self):
        logging.info(f"Downloading/Loading Model: {MODEL_NAME}")
        # Initialize tokenizer and model
        self.tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
        self.model = AutoModel.from_pretrained(MODEL_NAME)

        # Check if GPU is available
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model.to(self.device)
        self.model.eval()
        logging.info(f"Model loaded and moved to {self.device}")

        self.processed_ids = self._load_checkpoints()

    def _load_checkpoints(self) -> set:
        if os.path.exists(CHECKPOINT_PATH):
            with open(CHECKPOINT_PATH, 'r') as f:
                return set(line.strip() for line in f)
        return set()

    def _save_checkpoint(self, entry_id: str):
        with open(CHECKPOINT_PATH, 'a') as f:
            f.write(f"{entry_id}\n")
        self.processed_ids.add(entry_id)

```

```
    def clean_text(self, text: str) -> str:
        if not text:
            return ""
        # Basic cleanup
        text = text.replace('\n', ' ')
        text = ' '.join(text.split())
        return text

    def generate_vectors_batch(self, texts: List[str]) -> List[List[float]]:
        """
        Generates vector embeddings for a batch of texts using XLM-R.
        """
        if not texts:
            return []

        try:
            # Tokenize input text in batches
            inputs = self.tokenizer(
                texts,
                return_tensors="pt",
                truncation=True,
                max_length=512,
                padding=True
            ).to(self.device)

            # Generate embeddings
            with torch.no_grad():
                outputs = self.model(**inputs)

            # Mean pooling of the last hidden states
            attention_mask = inputs['attention_mask'].unsqueeze(-1).expand(outputs.
    last_hidden_state.size()).float()
            sum_embeddings = torch.sum(outputs.last_hidden_state * attention_mask, 1)
            sum_mask = torch.clamp(attention_mask.sum(1), min=1e-9)
            mean_pooled = sum_embeddings / sum_mask

            # Convert to list of lists of floats
            vectors = mean_pooled.cpu().numpy().tolist()
            return vectors

        except Exception as e:
            logging.warning(f"Vector generation failed for batch... Error: {e}")
            return [[] for _ in texts]

    def store_vectors_batch(self, entry_ids: List[str], vectors: List[List[float]]):
        """
        Appends a batch of generated vectors to the JSONL file.
        """
        with open(OUTPUT_PATH, 'a', encoding='utf-8') as f:
            for entry_id, vector in zip(entry_ids, vectors):
                if not vector:
                    continue
                record = {
                    "id": entry_id,
                    "vector": vector
                }
                f.write(json.dumps(record, ensure_ascii=False) + '\n')

    def process_dataset(self, batch_size: int = 32):
```

```
        logging.info(f"Loading dataset from {DATASET_PATH}...")
        try:
            with open(DATASET_PATH, 'r', encoding='utf-8') as f:
                data = json.load(f)
        except Exception as e:
            logging.error(f"Could not load JSON: {e}")
            return

        logging.info(f"Processing {len(data)} entries with batch_size={batch_size}. Found {len(self.processed_ids)} already processed.")

        batch_ids = []
        batch_texts = []

        for entry in tqdm(data):
            entry_id = entry.get('id')
            if not entry_id or entry_id in self.processed_ids:
                continue

            raw_text = entry.get('response', '')
            if not raw_text:
                self._save_checkpoint(entry_id)
                continue

            cleaned_text = self.clean_text(raw_text)

            batch_ids.append(entry_id)
            batch_texts.append(cleaned_text)

            if len(batch_ids) >= batch_size:
                vectors = self.generate_vectors_batch(batch_texts)
                self.store_vectors_batch(batch_ids, vectors)

                # Save checkpoints
                for b_id in batch_ids:
                    self._save_checkpoint(b_id)

                batch_ids = []
                batch_texts = []

        # Process remaining
        if batch_ids:
            vectors = self.generate_vectors_batch(batch_texts)
            self.store_vectors_batch(batch_ids, vectors)
            for b_id in batch_ids:
                self._save_checkpoint(b_id)

        logging.info("Pipeline finished successfully.")

if __name__ == "__main__":
    generator = VectorGenerator()
    try:
        generator.process_dataset()
    except KeyboardInterrupt:
        logging.info("Process interrupted by user. Progress saved.")
    except Exception as e:
        logging.error(f"Fatal error: {e}")
```

2 Vector Visualization

Purpose: We use this script to compute the cosine similarity between the generated vectors and visualize their relationships interactively using `pyvis`. It filters relationships based on a similarity threshold.

Listing 2: visualize_vector_relationships.py

```
import json
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
from pyvis.network import Network
import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(message)s')

VECTOR_FILE = "kurdish_medical_vectors.jsonl"
OUTPUT_HTML = "vector_relationships.html"
SIMILARITY_THRESHOLD = 0.85  # Adjust this to show more/fewer relationships

def load_vectors():
    ids = []
    vectors = []
    logging.info(f"Loading vectors from {VECTOR_FILE}...")
    try:
        with open(VECTOR_FILE, 'r', encoding='utf-8') as f:
            for line in f:
                data = json.loads(line)
                ids.append(str(data['id']))
                vectors.append(data['vector'])
    except FileNotFoundError:
        logging.error(f"{VECTOR_FILE} not found. Please run vector_generator.py first.
    ")
        return [], []
    return ids, np.array(vectors)

def build_graph():
    ids, vectors = load_vectors()
    if not ids:
        return

    logging.info(f"Loaded {len(ids)} vectors. Computing similarities...")

    # Compute cosine similarity between all vectors
    similarity_matrix = cosine_similarity(vectors)

    # Initialize pyvis network
    logging.info("Building interactive graph...")
    net = Network(height="100vh", width="100%", bgcolor="#222222", font_color="white")

    # Add nodes
    for i, node_id in enumerate(ids):
        # We only add nodes if they have at least one connection, to keep it clean.
        # But for now, we'll add all of them, or just the ones that connect.
        pass

    # Find relationships (edges) based on similarity using optimized NumPy operations
    edges = []
```

```
    connected_nodes = set()

    # Get the upper triangle of the similarity matrix (excluding diagonal)
    row_indices, col_indices = np.triu_indices_from(similarity_matrix, k=1)

    # Filter indices where similarity is above threshold
    valid_mask = similarity_matrix[row_indices, col_indices] >= SIMILARITY_THRESHOLD
    valid_rows = row_indices[valid_mask]
    valid_cols = col_indices[valid_mask]
    valid_sims = similarity_matrix[valid_rows, valid_cols]

    # Optional safety measure: limit max edges so the browser doesn't crash on massive graphs
    MAX_EDGES = 10000
    if len(valid_sims) > MAX_EDGES:
        logging.warning(f"Found {len(valid_sims)} edges, which might freeze the browser. Limiting to top {MAX_EDGES} strongest edges.")
        # Get indices of top MAX_EDGES similarities
        top_indices = np.argsort(valid_sims)[-MAX_EDGES:]
        valid_rows = valid_rows[top_indices]
        valid_cols = valid_cols[top_indices]
        valid_sims = valid_sims[top_indices]

    for r, c, sim in zip(valid_rows, valid_cols, valid_sims):
        edges.append((int(r), int(c), float(sim)))
        connected_nodes.add(int(r))
        connected_nodes.add(int(c))

    logging.info(f"Found {len(edges)} strong relationships (similarity >= {SIMILARITY_THRESHOLD}).")

    # Add only connected nodes to keep the graph readable
    for node_idx in connected_nodes:
        net.add_node(node_idx, label=f"Document {ids[node_idx]}", title=f"ID: {ids[node_idx]}")

    # Add edges
    for i, j, sim in edges:
        # Weight the edge based on similarity
        weight = (sim - SIMILARITY_THRESHOLD) / (1.0 - SIMILARITY_THRESHOLD) * 5
        net.add_edge(i, j, value=weight, title=f"Similarity: {sim:.2f}")

    # Configure physics for a nice Neo4j-like layout
    net.barnes_hut(gravity=-8000, central_gravity=0.3, spring_length=250)

    # Save the HTML file
    net.save_graph(OUTPUT_HTML)
    logging.info(f"✅ Graph saved to {OUTPUT_HTML}!")
    logging.info(f"Open {OUTPUT_HTML} in any web browser to view the relationships.")

if __name__ == "__main__":
    build_graph()
```

3 Setup Script

Purpose: We use this shell script to prepare the environment, install required Python dependencies from `requirements.txt`, create a `.env` template, and cache the XLM-R model.

Listing 3: setup.sh

```
#!/bin/bash

# KMC Vector Generation Setup Script

echo "🚀 Starting setup for KMC Vector Pipeline..."

# 1. Check for Python
if ! command -v python3 &> /dev/null; then
    echo "❌ Python3 is not installed. Please install it first."
    exit 1
fi

# 2. Install Requirements
echo "📦 Installing dependencies from requirements.txt..."
pip install --upgrade pip
pip install -r requirements.txt

# 3. Create .env template if it doesn't exist
if [ ! -f .env ]; then
    echo "📝 Creating .env template..."
    echo "MODEL_NAME=xlm-roberta-large" > .env
    echo "DATASET_PATH=../just_data_vector/kurdish_medical_corpus_kmc.json" >> .env
    echo "OUTPUT_PATH=kurdish_medical_vectors.jsonl" >> .env
fi

# 4. Download and Cache XLM-R Model
echo "🧠 Downloading and caching the 'xlm-roberta-large' model..."
python3 -c "
from transformers import AutoTokenizer, AutoModel
print('Downloading tokenizer...')
AutoTokenizer.from_pretrained('xlm-roberta-large')
print('Downloading model...')
AutoModel.from_pretrained('xlm-roberta-large')
print('Model cached successfully!')
"

echo "✅ Setup complete!"
echo "🚀 You can now run the pipeline: 'python vector_generator.py'"
```

4 Model Loader

Purpose: A dedicated script used to download and cache the `xlm-roberta-large` model from Hugging Face into the local environment before running the main pipeline.

Listing 4: load_model_gpu_1.sh

```bash
#!/bin/bash

# Load Model into Cache

echo -e "\033[1;33m[*] Action: Caching 'xlm-roberta-large' model...\033[0m"

# We use python to trigger the Hugging Face download
python3 -c "
from transformers import AutoTokenizer, AutoModel
print('Downloading tokenizer...')
AutoTokenizer.from_pretrained('xlm-roberta-large')
print('Downloading model...')
AutoModel.from_pretrained('xlm-roberta-large')
print('Done!')
"

echo -e "\033[1;32m[SUCCESS] Model 'xlm-roberta-large' is cached successfully!\033[0m"
```

5 Requirements

Purpose: Defines the required Python packages for running the vector generation and visualization pipeline.

Listing 5: requirements.txt

```
transformers
torch
tqdm
scikit-learn
pyvis
```

6 Environment Variables

Purpose: The configuration settings used by the vector generator.

Listing 6: .env

```
MODEL_NAME=xlm-roberta-large
DATASET_PATH=../just_data_vector/kurdish_medical_corpus_kmc.json
OUTPUT_PATH=kurdish_medical_vectors.jsonl
```